



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Sistemas de Bases de Dados
Relatório: Trabalho Prático 2

Turma: 51D

Nome: Miguel Alcobia Número: 50746

Email: A50746@alunos.isel.pt

Nome: Tomás Salvador Número: 50766

Email: A50766@alunos.isel.pt

Professor: António Teófilo

Data: 5 de janeiro de 2025

Índice

1. - Conceção	1
1.1. - Modelo entidade associação, referindo os pressupostos assumidos	1
1.2. - Modelo relacional, domínios e chaves (candidatas, primárias e estrangeiras)	3
1.3. - Restrições de integridade aplicacional	8
2. - Concretização	9
2.1. - Criação do modelo físico (criação de tabelas e vistas) através de um <i>script</i> SQL DDL	9
2.2. - Carregamento de dados de teste (mínimo 3 registos por tabela) realizado por um <i>script</i> SQL DML	10
2.3. - Codificação de um <i>script</i> SQL DDL para eliminar o modelo físico da base de dados	10
3. - Segunda Parte do Trabalho Prático - SQL	11
3.1. - Administrador	11
3.2. - Cliente	13
3.3. - Condutor	14
3.4. - Funcionário	14
3.5. - Gerente	15
4. - Segunda Parte do Trabalho Prático - <i>Web App</i>	17
Conclusões	19
Bibliografia	20

Índice de Figuras

Figura 1 – Modelo de Entidade - Associação (1ª Parte).....	1
Figura 2 - Modelo de Entidade - Associação (2ª Parte).....	11

1. - Conceção

1.1. - Modelo entidade associação, referindo os pressupostos assumidos

O modelo Entidade-Associação é um modelo utilizado para modelar dados, estes modelos identificam e descrevem os elementos do mundo real (entidades) e as relações entre eles (associações). Esta abordagem permite organizar e descrever cenários de dados de um sistema, de forma estruturada e intuitiva. Desta forma o modelo Entidade-Associação, facilita a compreensão e a comunicação entre os observadores do modelo, permitindo uma descrição mais clara e detalhada dos dados através das entidades e das suas associações.

A fase inicial deste trabalho, passou pela análise do texto facultado no enunciado de forma a extrair a informação necessária para organizar as entidades e os seus atributos, e por fim identificar as relações entre elas. Com esta informação seleccionada passou-se ao desenvolvimento do modelo Entidade-Associação, com todas as entidades, atributos e relações apresentadas. O modelo desenvolvido pode ver-se na seguinte figura:

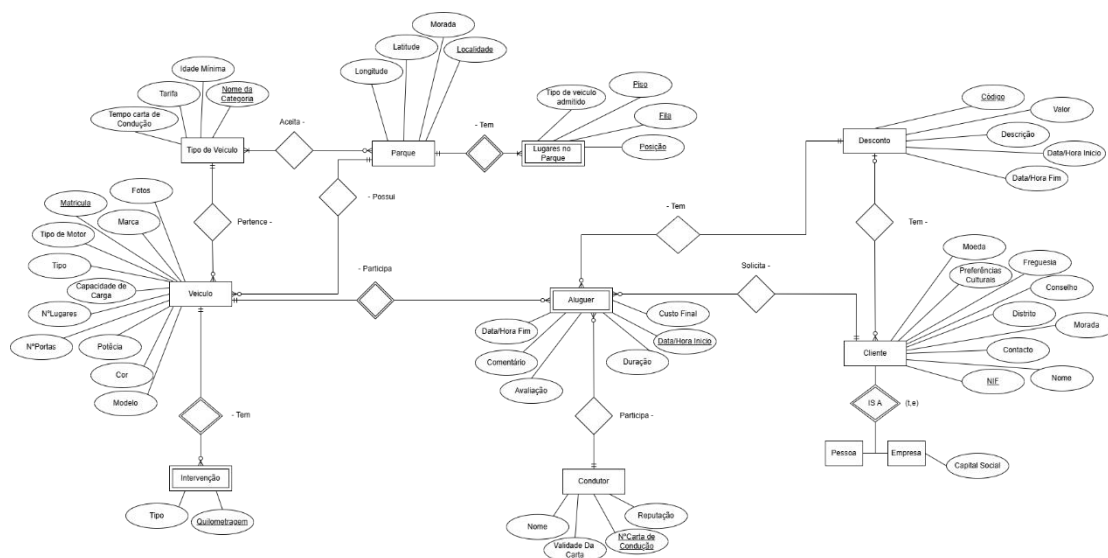


Figura 1 – Modelo de Entidade - Associação (1ª Parte)

Para o desenvolvimento do modelo, foram primeiro definidas todas entidades: Conductor, Aluguer, Desconto, Veiculo, Tipo de Veiculo, Intervenção, Parque, Lugares no Parque e Cliente, que pode ser uma Pessoa ou uma Empresa. Em seguida, foram definidos os atributos e as chaves primárias e parciais de cada entidade.

As chaves primárias são atributos que identificam de forma única e inequívoca uma

entidade no modelo. Permite garantir que cada instância da entidade seja distinta, como é o caso do NIF relativo a uma pessoa. Por exemplo, uma pessoa pode ter um nome e uma idade e essas “informações ” não serem suficientes para a sua identificação, pois podem existir outras pessoas com informações iguais. Já o NIF é um valor único associado a cada pessoa, e não podem existir duas pessoas com o mesmo NIF.

Já as chaves parciais são atributos que, sozinhos, não conseguem identificar uma instância de uma entidade, mas se forem combinados com a chave primária de outra entidade que lhe seja associada, formam uma identificação única.

A entidade Tipo de Veículo foi criada para se poderem classificar os veículos de acordo com a sua categoria, pois desta forma é possível organizar veículos em grupos lógicos, como familiar, comercial, motociclo, etc.... Esta organização facilitará a gestão de regras específicas, como quem vai poder alugar cada tipo de veículo, de acordo com a Idade mínima e/ou tempo com carta de condução. Para identificação do Tipo de veículo a chave primária selecionada foi o “Nome da Categoria”.

Como os Clientes podiam ser uma Pessoa ou uma Empresa, recorreu-se a uma generalização para representar as especializações da entidade Cliente, que neste caso são as entidades Pessoa e Empresa. O uso das generalizações permitem evitar redundâncias, uma vez que tornam possível a representação das características comuns numa entidade pai e de características específicas nas entidades filhas. Como Pessoa e Empresa são sub entidades da interface Cliente, são identificadas pela sua chave primária (NIF).

Esta generalização é útil pois permitiu tratar os clientes de forma genérica, mas, ao mesmo tempo, manter os detalhes específicos de Pessoas e de Empresas, garantindo uma maior flexibilidade no modelo.

O modelo possui também três entidades fracas. As entidades fracas são entidades que dependem de outra entidade para existirem, ou seja, não possuem uma chave primária própria, mas utilizam uma chave parcial combinada com a chave primária da sua entidade forte para serem identificadas inequivocamente. As entidades fracas representam conceitos que não têm significado se estiverem fora do contexto da sua entidade forte.

Neste modelo a entidade Lugares Parque é uma entidade fraca uma vez que depende da entidade forte Parque para existir. Pois a identificação de um lugar no parque só faz sentido dentro do contexto específico de um determinado parque.

A Intervenção é outra entidade fraca, associada à entidade forte Veículo. A intervenção só existe em relação a um veículo específico, sendo por isso necessário combinar informações da intervenção com o veículo para identificar a intervenção de forma única.

A entidade Aluguer também é uma entidade fraca, pois a sua existência depende diretamente da entidade forte Veículo. Um Aluguer só faz sentido no contexto de um veículo específico, ou seja, não é possível identificar um aluguer de forma inequívoca sem o associar a um veículo. Por isso, foi utilizada uma chave parcial, com a Data/Hora Início combinada com a Matrícula do veículo, desta forma já é possível identificar um aluguer de forma única.

A entidade Desconto foi criada para representar um desconto que pode ser aplicado ao custo de um Aluguer específico. Esta entidade está associada tanto à entidade Cliente quanto à entidade Aluguer, uma vez que, os descontos podem depender do cliente, e são aplicados a um aluguer específico no momento da sua concretização.

As entidades restantes são: Condutor, Parque e Veiculo que são identificadas pelas chaves primárias N° da Carta de condução, Matrícula e Localidade, respetivamente. Como no enunciado está escrito que “só pode existir um parque por localidade”, a Localidade é suficiente para identificar o parque, daí a escolha desse atributo para chave primária.

1.2 - Modelo relacional, domínios e chaves (candidatas, primárias e estrangeiras)

Nesta fase foi desenvolvido o modelo relacional, que é um modelo lógico de organização de dados que utiliza tabelas ou relações para representar as informações e as suas interdependências. Cada tabela é composta por linhas (ou no caso das relações são compostas por tuplos), que representam instâncias de dados, e colunas, que representam os atributos de uma entidade ou relação. O modelo utiliza chaves primárias para identificar cada linha de forma única e chaves estrangeiras para estabelecer relações entre as tabelas diferentes. Desta forma, oferece uma abordagem estruturada para armazenar e manipular os dados, garantindo integridade e permitindo consultas e operações através de álgebra relacional e SQL, como vamos analisar mais á frente.

As chaves estrangeiras nas relações são atributos ou conjuntos de atributos que estabelecem ligações entre tabelas relacionadas, fazendo a ligação de uma relação à chave primária de outra relação. Desta forma, as chaves ajudam a garantir a integridade referencial no modelo.

Como se pode ver no modelo da fig.1, os tipos de associações binárias nele presentes são:

- Associação binária 1:N (1-1 : 0-N) - significa que uma entidade do lado "1" pode estar associada a várias entidades do lado "N" (ou não estar associado a nenhuma), enquanto cada entidade do lado "N" só pode estar associada a uma entidade do lado "1". O segundo lado referencia o primeiro lado, ficando com a sua chave primária como atributo e como chave estrangeira.
- Associação binária 1:N (1-1 : 1-N) - significa que uma entidade do lado "1" pode estar associada a várias entidades do lado "N" (no mínimo 1), enquanto cada entidade do lado "N" só pode estar associada a uma entidade do lado "1". O segundo lado referencia o primeiro lado, ficando com a sua chave primária como atributo e como chave estrangeira.
- Associação binária M:N (M-1 : 0-N) - significa que uma entidade (M) está associada a uma única instância de outra entidade (1), e essa instância pode-se associar a várias do outro lado (N). Esta associação cria um esquema relacional próprio que fica com as chaves primárias de ambos os lados, como atributos e como chaves candidatas e estrangeiras.
- Associação binária 1:N (1-0 : 0-N) - significa que uma entidade do lado "1" pode estar associada a várias entidades do lado "N" e cada entidade do lado "N", se existir só pode estar associada a uma entidade do lado "1". Esta associação cria um esquema relacional próprio que fica com as chaves primarias de ambos os lados como atributos. A sua chave candidata vai ser igual a chave primaria do segundo lado, e a sua chave estrangeira, vai ser igual as chaves primarias de ambos os lados da relação.

Estas foram as relações criadas para o modelo relacional:

Cliente (nome, numeroFiscal, contacto, morada, distrito, concelho, freguesia, preferenciasLinguisticasECulturais, moeda)

- Chaves Candidatas = {numeroFiscal}

Alugar (dataHoraInicio, dataHoraFim, duracao, custoFinal, avaliacao, numeroFiscal, numeroCartaConducao, codigo, matricula, comentario)

- Chaves Candidatas = {matricula, dataHoraInicio}

- Chaves Estrangeiras = {matricula, numeroFiscal, numeroCartaConducao, codigo}

Veiculo (matricula, marca, modelo, fotos, cor, tipo, numeroLugares, numeroPortas, capacidadeCarga, tipoMotor, potencia, localidade, nomeCategoria)

- Chaves Candidatas = {matricula}
- Chaves Estrangeiras = {localidade, nomeCategoria}

Intervenção (tipo, quilometragem, matricula)

- Chaves Candidatas = {quilometragem, matricula}
- Chaves Estrangeiras = {matricula}

Condutor (nome, numeroCartaConducao, validadeCartaa, reputacao)

- Chaves Candidatas = {numeroCartaConducao}

Parque(localidade, morada, latitude, longitude)

- Chaves Candidatas = {localidade}

LugarParque(piso, fila, posicao, tipoVeiculoAdmitido, localidade)

- Chaves Candidatas = {localidade, piso, fila, posicao, tipoVeiculoAdmitido}
- Chaves Estrangeiras = {localidade}

Cliente_Desconto(numeroFiscal, codigo)

- Chaves Candidatas = {numeroFiscal}
- Chaves Estrangeiras = {numeroFiscal} e {codigo}

Parque_Tipo(localidade, nomeCategoria)

- Chaves Candidatas = {localidade, nomeCategoria}
- Chaves Estrangeiras = {localidade} e {nomeCategoria}

Em seguida foram também definidos os domínios que representam o conjunto de valores possíveis e válidos que um atributo pode assumir dentro de um banco de dados, garantindo que esses valores sejam consistentes e adequados a cada contexto. Cada domínio é composto por valores atômicos, ou seja, valores indivisíveis no contexto da aplicação, como números, textos ou datas. A utilização dos domínios é essencial para garantir a integridade dos dados e facilitar o controlo e a validação das informações que vão ser armazenadas.

Cliente

- **nome:** VARCHAR(255)
- **numeroFiscal:** CHAR(9)
- **contacto:** CHAR(9)
- **morada:** VARCHAR(255)
- **distrito:** VARCHAR(100)
- **concelho:** VARCHAR(100)
- **freguesia:** VARCHAR(100)
- **preferenciasLinguisticasECulturais:** TEXT - Lista com as preferências do cliente
- **moeda:** char(3) - Abreviatura das unidades monetárias

Condutor

- **nome:** VARCHAR(255)
- **numeroCartaConducao:** CHAR(12) - Após alguma pesquisa, o grupo não encontrou nenhuma informação sobre o número de dígitos da Carta de Condução, encontrando exemplos que variavam entre 9 e 12 dígitos. Por este motivo, optou-se pelos 12 dígitos.
- **validadeCarta:** DATE
- **reputacao:** DECIMAL(3,2) - 0.00 a 5.00, por exemplo.

Parque

- **localidade:** VARCHAR(100)
- **morada:** VARCHAR(255)
- **latitude e longitude:** DECIMAL(9,6)

LugarParque

- **localidade** VARCHAR(100)
- **piso:** INTEGER
- **fila:** CHAR(3) - fila A, B, C, por exemplo.
- **posicao:** INTEGER
- **tipoVeiculoAdmitido:** VARCHAR(50)

Veiculo

- **matricula: CHAR(8)** - Optou-se pelo formato de duas letras, dois dígitos, duas letras.
- **marca: VARCHAR(50)**
- **modelo: VARCHAR(50)**
- **fotos: LONGTEXT** - As fotos serão guardadas em Base64.
- **cor: VARCHAR(20)** - Por exemplo, Vermelho, Azul, Azul Metálico etc.
tipo: VARCHAR(50).
- **numeroLugares: INTEGER**
- **numeroPortas: INTEGER**
- **capacidadeCarga: DECIMAL(6,2)** - A capacidade será expressa em quilogramas.
- **tipoMotor: VARCHAR(20)** - Por exemplo, Elétrico, Híbrido, Combustão.
- **potencia: DECIMAL(5,2)** - A capacidade será expressa em Kw.

TipoVeiculo

- **tempoCarta: INTEGER**
- **tarifa: DECIMAL(10,2)** - preço por por dia.
- **idadeMinima: INTEGER**
- **nomeCategoria: VARCHAR(50)**

Aluguer

- **dataHoraInicio: DATETIME**
- **dataHoraFim: DATETIME**
- **duracao: DECIMAL(6,2)** - Decimal, para caso o aluguer não seja para um dia inteiro.
- **custoFinal: DECIMAL(10,2)**
- **codigo: CHAR(6)** - Opcional, pode ser NULL.
- **avaliacao: DECIMAL(3,2)** - Avaliação de 0.00 a 5.00.
- **comentário: VARCHAR(150)** - Por exemplo, "Adorei.", "Gostei.", "Não vou Voltar." ou algo mais detalhado.
- **numeroFiscal CHAR(9)**
- **numeroCartaConducao CHAR(12)**
- **codigo CHAR(6)**
- **matricula CHAR(8)**

Intervenção

- **tipo: VARCHAR(50)** - Por exemplo, Revisão, Reparação, Troca de Óleo.
- **quilometragem: INTEGER**
- **matricula CHAR(8)**

Desconto

- **codigo: CHAR(6)**
- **valor: DECIMAL(3,2)** - O valor do desconto será expresso entre 0.00 e 1.00.
- **descricao: VARCHAR(255)** - Descrição do que é a intervenção e do seu eventual motivo.
- **dataHoraInicio: DATETIME**
- **dataHoraFim: DATETIME**

1.3. - Restrições de integridade aplicacional

Nesta fase do trabalho foram também definidas as RIA (Restrições de Integridade Aplicacional), que são regras que as relações do conjunto do Esquema de Relações devem obedecer, mas que não se conseguem definir apenas através das restrições já mencionadas anteriormente. Estas restrições são normalmente implementadas ao nível da aplicação.

Para definir estas RIA procedeu-se á leitura do enunciado, e foram seleccionadas as seguintes restrições:

- O aluguer deve ter uma duração entre 1 hora e 14 dias (336 horas)
- A data de início do aluguer deve estar dentro do período de validade do desconto.
- O custo final deve ser calculado com base na moeda preferida do cliente, aplicando uma conversão, se necessário.
- A validade da carta de condução deve ser maior que a data de término do aluguer.
- Certos tipos de veículos só podem ser alugados se o condutor tiver mais de 25 anos ou possuir carta de condução há mais de 2,5 anos.
- A reputação do condutor deve ser atualizada com base no feedback de cada aluguer (ex.: avaliação negativa reduz a reputação).
- Um lugar de estacionamento só deve aceitar tipos de veículos que sejam

compatíveis com ele (ex.: uma vaga para motocicletas não pode ser usada para veículos comerciais).

- Um cliente só pode registrar um comentário se escolher uma avaliação associada (adorei, gostei, não vou voltar).
- Cada aluguer pode ter, no máximo, uma avaliação registrada.
- Após o pagamento de um aluguer, se foi usado um desconto, este deve ser desassociado do cliente.

2. - Concretização

2.1. - Criação do modelo físico (criação de tabelas e vistas) através de um *script* SQL DDL

Com a parte dos domínios já bem estruturada e definida, a criação do modelo em MySQL foi facilmente implementada. O grupo usou o programa MySQL Workbench para mexer na base de dados através de uma interface gráfica, facilitando o fluxo de trabalho.

Criou-se uma base de dados chamada `sbd_a50746_405766` e para cada tabela utilizou-se o comando `CREATE TABLE` com os atributos definidos com os respetivos domínios como referido no ponto anterior. Para a definição das chaves primárias e estrangeiras utilizaram-se `CONSTRAINTS` da seguinte forma:

- Para Chaves Primária:
`CONSTRAINT <nome da restrição> PRIMARY KEY (<atributos que formam chave primária>)`
- Para Chaves Estrangeira:
`CONSTRAINT <nome da restrição> FOREIGN KEY (<atributos que formam a chave estrangeira >) REFERENCES <Tabela de onde vêm esses atributos que formam a FK>(<atributos que formam a chave estrangeira>)`

A ordem do carregamento dos dados teve de ter em atenção as dependências entre tabelas, que são criadas pelas chaves estrangeiras.

2.2. - Carregamento de dados de teste (mínimo 3 registos por tabela) realizado por um *script* SQL DML

Para esta fase utilizou-se o comando `INSERT INTO` seguido de `VALUES` em cada tabela para que fossem carregados os dados criados.

Após a criação da tabela `Aluguer`, usou-se o comando `UPDATE` e `WHERE EXISTS` para atualizar o valor da reputação de um condutor com a média das avaliações que recebeu, no caso do condutor já possuir alugueres associados a ele.

Depois criaram-se vistas para cenários que o grupo considerou interessantes, dado o contexto do trabalho. As vistas criadas foram:

- **`vw_Clientes`** - Mostra a informação dos clientes, especificando o tipo deles (Pessoa ou Empresa) e caso seja Empresa também mostra o seu capital.
- **`vw_AlugueresDetalhados`** - Mostra informações completas dos alugueres realizados por um cliente específico.
- **`vw_VeiculosPorTipo`** - Mostra os veículos disponíveis por tipo e as informações associadas a cada tipo, como a potência média e capacidade de carga média.
- **`vw_DescontosAtivos`** - Mostra os descontos ativos atualmente.
- **`vw_VeiculosParque`** - Mostra a que parque está associado cada veículo.
- **`vw_ReputacaoCondutor`** - Mostra a reputação do Condutor, tendo em conta a média das suas avaliações.

2.3. - Codificação de um *script* SQL DDL para eliminar o modelo físico da base de dados

Para eliminar o modelo físico, criou-se mais um *script* onde se começou por recorrer aos comandos `SELECT`, `FROM` e `WHERE` para obter uma tabela que associa uma chave estrangeira à tabela onde esta se encontra. Desta forma, a criação dos comandos `ALTER TABLE <Tabela> DROP FOREIGN KEY <chave estrangeira>` ficou facilitada.

Em seguida, após a eliminação das chaves estrangeiras, utilizou-se o comando `DROP TABLE IF EXISTS <tabela>` para eliminar as tabelas do modelo, acabando assim por eliminar o modelo por completo da base de dados. Por fim, com o objetivo de realizar testes durante a concretização do trabalho, também se recorreu ao comando `DROP DATABASE IF EXISTS <base de dados>` para eliminar a base de dados.

3. - Segunda Parte do Trabalho Prático - SQL

Esta segunda fase do trabalho tem como objetivo desenvolver uma aplicação na Web com uma interface mais amigável para o utilizador para a solução apresentada na primeira fase do trabalho.

Visto que da primeira parte do enunciado para a segunda surgiram novas observações e requisitos para o projeto, foi necessário fazer alguns ajustes na solução apresentada na primeira fase do trabalho. As alterações foram as seguintes:

- **Lugar** passa a ter um atributo "estado" e matricula
- **Aluguer** passa a ter um atributo *status*
- **Intervencao** passa a ter um atributo dataIntervencao (passa a ser a Chave Primária com a matricula em vez da quilometragem)
- **Veiculo** e **Lugar** passam a ter uma associação 1:0-0:1

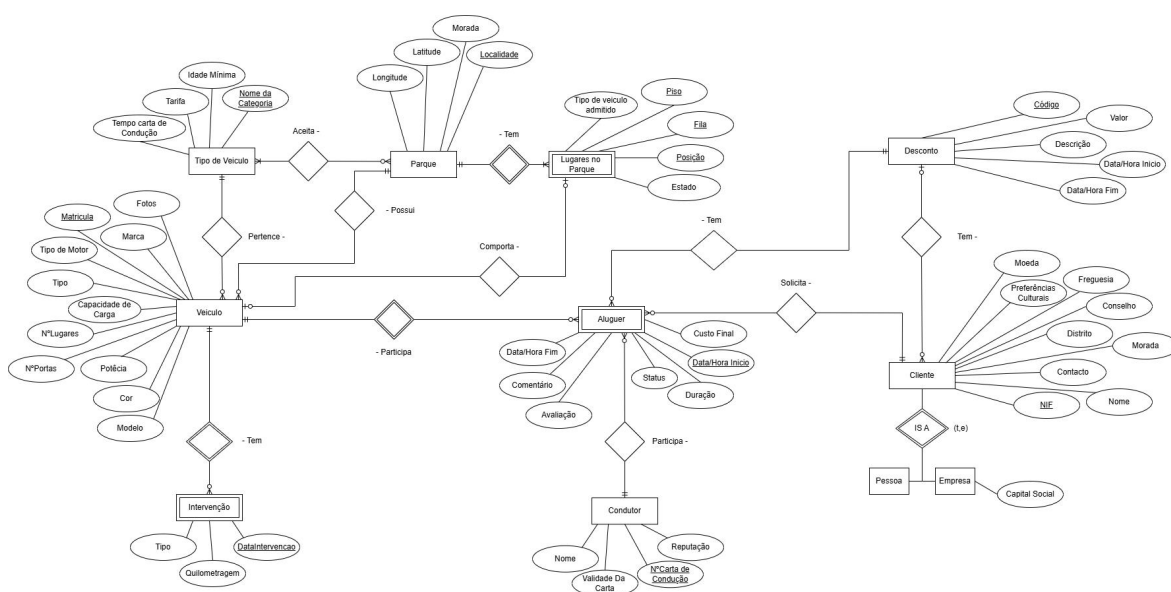


Figura 2 - Modelo de Entidade - Associação (2ª Parte)

A ligação entre a base de dados em MySQL e a linguagem JAVA foi feita através de JDBC e a implementação na Web foi efetuada com o auxílio de Servlets e JSPs.

3.1. - Adminstrador

Para o administrador começou-se por elaborar os comandos SELECT necessários para criar e atualizar Clientes e Condutores. Depois desenhou-se a interface de forma a pedir os inputs para preencher esses comandos. Por exemplo, para criar um Cliente é

preciso fornecer o nome, telefone, morada (Rua e número), distrito, concelho, freguesia, preferências linguísticas, moeda, tipo de cliente (Pessoa ou Empresa) e capital Social, caso seja uma Empresa.

Estes inputs são enviados para a função `adicionarCliente()` que faz um INSERT na tabela Cliente e depois verifica o tipo de Cliente para saber se faz outro INSERT na tabela das Pessoas ou das Empresas. Logo após a criação do Cliente, também é realizado um INSERT na tabela Cliente_Desconto, para este novo cliente ter na sua posse o desconto D00001, que é um desconto para novos clientes.

Para atualizar um campo no Cliente, pergunta-se o número Fiscal do cliente que se pretende atualizar, o campo a renovar a informação e o novo valor desse campo. Esta informação é passada à função `updateCampoCliente()`, que verifica qual o campo que é suposto atualizar, pois caso sejam o capital social ou o número Fiscal têm funções próprias, porque são valores que também têm de ser atualizados nas tabelas Pessoa e Empresa.

Utilizou-se o mesmo raciocínio para criar e atualizar os **condutores** e os **veículos**.

O grupo viu a necessidade de criar uma alínea extra para arrumar os veículos. Visto que estes não ficavam efetivamente em nenhum parque até que um condutor entregasse o veículo. Portanto, para contornar este problema, depois do administrador inserir a matrícula do veículo que deseja estacionar, a informação é enviada para a função `arrumarVeiculo()` e verifica na tabela `veiculo`, qual é a categoria e a localidade daquele veículo. Depois, com essa informação, procura um lugar livre dessa categoria no parque dessa localidade `obterLugaresLivresLocalidadeCategoria()`. Ao encontrar um lugar, passa-se o seu estado de “livre” para “ocupado” e associa-se o veículo a esse lugar na tabela `Veiculo_LugarParque`.

Para as alíneas de exportar e importar o registo cronológico de um veículo, criaram-se as funções `exportJSON()` e `importJSON()`.

No `exportJSON()`, consultam-se os dados completos de um veículo, após o utilizador inserir a respetiva matrícula e o caminho onde quer que o ficheiro exportado fique. Para passar a informação para um ficheiro JSON, faz-se uma consulta SQL numa vista chamada `vw_fullHistoryCar`, que tem o histórico completo dos veículos existentes na base de dados, incluindo eventos (Intervenções/Alugueres), quilometragem e avaliações (no caso dos Alugueres). Os resultados são percorridos e armazenados como objetos JSON num `JSONArray`, sendo que cada objeto corresponde a um evento (Intervenção ou Aluguer). O ficheiro é guardado com um nome único, gerado pela

combinação da matrícula e a data/hora.

Desta forma, garante-se a preservação dos dados num formato estruturado, com boa legibilidade.

Já no `importJSON()`, realiza-se o processo inverso: lê um arquivo JSON que tem as informações de eventos (Intervenção ou Aluguer) e insere os dados no banco de dados. O arquivo é lido e convertido para um `JSONArray`, onde cada objeto JSON é processado individualmente. Dependendo do tipo de evento identificado no campo "evento", a função chama a função `adicionarIntervencaoJSON()` ou `adicionarAluguerJSON()`, para gravar os dados na tabela correspondente.

3.2. - Cliente

Para possibilitar que o cliente efetuasse uma reserva, é pedido para que este insira os dados que caracterizam a reserva, que são os mesmos do Aluguer, com a diferença de que a reserva não tem um veículo nem um condutor associados a ela.

É feita uma consulta à vista `vw_DescontosAtivosCliente` que mostra os descontos ativos (que as datas de validade ainda são válidas) associados ao cliente, segundo o número fiscal. Caso o cliente use o desconto, esta associação é eliminada, visto que na solução proposta um cliente só pode ter um desconto associado.

Após a escolha do desconto é calculado o custo final. Para isto obtém-se a moeda do cliente, o valor que esta tem associado na tabela de câmbio (que tem por moeda base o dólar americano. Os valores de câmbio foram consultados no site do Banco de Portugal a 17 de Dezembro de 2024) e a tarifa que a categoria escolhida possui. Com estes valores calcula-se o custo final e, caso exista, aplica-se o desconto escolhido pelo cliente.

Para consultar as suas Reservas e Alugueres, o cliente deve inserir o seu NIF e a partir dele são realizadas dois `SELECTS` distintos: um na tabela das Reservas e outro na dos Alugueres e é lhe apresentada a informação.

Para consultar a sua reputação (média das avaliações que deu) e os descontos que tem ativos na sua conta, o cliente deve inserir o número fiscal para depois se efetuarem os `SELECTS` nas respetivas tabelas que possuem a informação pretendida (vistas `vw_clientecomreputacao` e `vw_DescontosAtivosCliente`).

Por último, o grupo viu a necessidade de criar mais uma alínea para o cliente para que este conseguisse avaliar os alugueres. Para tal, o cliente insere o seu NIF e é lhe

apresentada uma lista dos alugueres que lhe estão associados que têm status “Concluído” e avaliação e comentário a NULL.

O cliente escolhe qual o aluguer que quer avaliar, dá uma nota e escreve um comentário e o aluguer é atualizado com a ajuda da função `atualizarComentarioAvaliacao()` que recebe esses parâmetros e usa o comando UPDATE para atualizar o aluguer escolhido.

3.3. - Condutor

O levantamento de veículos dá-se após o condutor inserir o número da sua carta de condução. Com o número da carta, a função `verReservasPendentesCondutor()` realiza um SELECT na tabela Aluguer para extrair os alugueres que têm status “Pendente” e o número da carta de condução do condutor associado. Esses alugueres são apresentados em forma de lista e o condutor escolhe qual dos alugueres pretende dar início.

A partir do aluguer escolhido, obtém-se a matrícula do veículo e com esta informação desocupa-se o lugar onde este estava, passando o seu estado para “livre”, e desassocia-se o veículo do lugar. Por fim, passa-se o status do aluguer de “Pendente” para “Em andamento”.

Para a entrega de Veículos a lógica é semelhante à de quando o Administrador os arruma depois da sua criação, com a diferença de que o condutor, além da matrícula do veículo, também precisa de explicitar o parque no qual quer arrumar o veículo. Após a entrega do veículo o status do aluguer passa de “Em andamento” para “Concluído”.

3.4. - Funcionário

Para atribuir veículos a uma reserva, é apresentada uma lista das reservas existentes na qual o funcionário escolhe aquela a que pretende atribuir um veículo e um condutor.

Para a escolha do veículo, o funcionário recebe uma lista dos que estão disponíveis na localidade escolhida pelo cliente quando fez a reserva e que sejam da categoria requisitada. Após a escolha do veículo, segue-se a escolha do condutor, onde o funcionário escolhe o condutor que realizará o aluguer.

No fim, os dados da reserva e os das escolhas efetuadas pelo funcionário são passadas à função `adicionarAluguer()`, que faz um INSERT com os dados recebidos na tabela Aluguer, e a reserva é eliminada a partir de um comando DELETE.

Para localizar um veículo, o funcionário deve inserir a matrícula do respetivo veículo

e essa informação é passada à função `consultarLocalizarVeiculo()` que realiza um `SELECT` na vista `vw_LocalizacaoVeiculos` a partir da matrícula.

Para criar uma intervenção, o utilizador deve inserir os atributos que caracterizam a Intervenção e a função `adicionarIntervencao()` faz um `INSERT` na tabela das intervenções com os dados inseridos.

A aliena da pesquisa do Autocomplete não foi feita na interface de Web por falta de tempo; contudo, esboçou-se a ideia do conceito na interface de consola.

É apresentada ao utilizador uma lista de todos os clientes (resultante de um `SELECT` na vista `vw_clientescomreputacao`, onde se retornam pares de NIF e nome dos clientes), depois pede-se que insira uma sequência de caracteres e dê “enter”. A função `filtrarSugestoes()`, filtra os clientes sugeridos consoante a sequência de caracteres que o utilizador já inseriu. No momento em que apenas um cliente coincide com a entrada do utilizador, é retornada a informação desse mesmo cliente.

Para atribuir descontos, ou não, ao cliente, o funcionário deve inserir o número Fiscal do cliente em questão para então se verificar se este está apto para receber descontos. Verifica-se a reputação do cliente: caso esteja igual a 0 ou então igual ou superior a 3, o cliente pode receber um desconto, desde que não tenha nenhum desconto ativo por usar (recorre-se à tabela `vw_DescontosAtivos`). Neste caso, o cliente terá de gastar o desconto já existente para então poder receber um novo.

Por último, o funcionário tem como opção saber quem conduzia um determinado veículo, numa determinada data. Para saber a identidade do condutor, o funcionário tem de inserir a matrícula do veículo e a data em causa. A partir destes dados, a função `identificarCondutor_Matricula_Data()` efetua um comando `SELECT` na tabela da vista `vw_condutorporaluguer` e retorna as informações do condutor e a data inicial e final do aluguer em execução na data pretendida.

3.5. - Gerente

O gerente tem como primeira opção consultar os históricos de um determinado veículos, para isso basta inserir a matrícula do respetivo veículo. O grupo achou por bem criar dois tipos de histórico, em duas vistas diferentes:

A primeira foi a que o se usou nesta alínea (`vw_historyCar`) que tem apenas a

informação da matrícula, o tipo de evento (Intervenção/Aluguer), a data do evento (no caso de ser um Aluguer trata-se da data final) e a avaliação, que no caso das Intervenções se encontra a NULL.

Já a outra vista (vw_fullHistoryCar) contem a mesma informação que a anterior, mais os restantes atributos de cada tipo de evento que não estavam presentes na anterior. Esta vista foi utilizada para as alíneas de JSON, porque era necessário ter toda esta informação para preencher a tabela das Intervenções e dos Alugueres ao importar um ficheiro.

As restantes opções do gerente, limitavam-se a apresentações de rankings e por consequência a abordagem foi bastante semelhante para todos eles, mudando apenas os inputs pedidos ao utilizador e a ordem pela qual os dados eram organizados no comando SELECT.

Para o primeiro o objetivo era apresentar as 3 marcas com menor lucro. Neste caso não era necessário pedir nenhum input ao utilizador, bastou apenas recorrer à função ranking3MenorLucro() que efetua um comando SELECT onde se combinam os dados de quatro tabelas: Aluguer (a), Cliente (cl), Cambio (c), e Veiculo (v). Ela calcula o lucro em dólares dividindo o custo final de cada aluguer pelo valor da moeda do cliente em relação ao dólar (obtido na tabela Cambio). A soma desses lucros é agrupada por marca ao usar SUM(a.custoFinal / c.valorPorDolar). O resultado é então ordenado por ordem crescente (ORDER BY lucroEmDolares ASC) e limitado aos três primeiros registos (LIMIT 3).

Para o segundo ranking, assim como no terceiro, o grupo optou por dar liberdade ao utilizador de escolher o intervalo de tempo que o ranking deveria cobrir. Por esta razão, nestas alíneas é pedido que o utilizador escolha duas datas e depois para cada alínea recorre-se a um comando SELECT diferente:

Na segunda, combinam-se os dados das tabelas Aluguer (a) e Veiculo (v), unindo-as pela coluna matrícula. Calcula-se a média das avaliações dos alugueres (AVG(a.avaliacao)) para cada combinação de marca e modelo, excluindo registos onde a avaliação é nula (a.avaliacao IS NOT NULL). Além disso, filtra os registos por um intervalo de datas (a.dataHoraInicio >= 'dataInicio' AND a.dataHoraInicio <= 'dataFim'). Os resultados são agrupados por marca e modelo (GROUP BY v.marca, v.modelo), ordenados pela média das avaliações em ordem decrescente (ORDER BY mediaAvaliacao DESC), e limitados aos cinco melhores registos (LIMIT 5).

Já na terceira, somam-se as quilometragens de um veículo (SUM(quilometragem) AS

quilometragemTotal). Os registos são filtrados para incluir apenas intervenções realizadas no intervalo pretendido (`dataIntervencao >= 'dataInicio' AND dataIntervencao <= 'dataFim'`) e são ordenados por ordem crescente de quilometragem total (`ORDER BY quilometragemTotal ASC`), garantindo que os veículos com menor quilometragem sejam priorizados. Os resultados são limitados aos 10 primeiros registos (`LIMIT 10`).

Já na última aliena o objetivo era organizar os 100 clientes num ranking com melhor reputação e com a possibilidade de, eventualmente, filtrar os dados pela freguesia. Para tal, o utilizador pode, ou não, escrever no campo que lhe é apresentado a freguesia pela qual pretende filtrar o ranking. Caso não escreva a freguesia assume o valor NULL.

O `SELECT` deste caso utiliza a vista `vw_clientecomreputacao` para obter as colunas `nome`, `reputacao`, e `freguesia`. Se um parâmetro de freguesia específico for fornecido (`freguesia != null && !freguesia.isEmpty()`), a consulta retorna apenas os clientes dessa freguesia. Caso contrário, todos os registos da visão serão considerados.

Os resultados são então ordenados por reputação em ordem decrescente (`ORDER BY reputacao DESC`) e a saída é limitada aos 100 registos mais relevantes (`LIMIT 100`).

4. - Segunda Parte do Trabalho Prático - *Web App*

O processo inicia-se com a página JSP, que funciona como a interface da aplicação. A JSP contém um formulário HTML onde o utilizador insere informações, como o NIF ou o número da carta de condução, em caixas de texto, entre outros.

Cada campo deste formulário possui um identificador único (atributo `name`), que será utilizado para identificar os dados no servidor. Quando o utilizador submete o formulário através de um botão na página, os dados são transmitidos para o servidor através de uma pedido HTTP, seja ele do tipo POST ou GET, dependendo da operação realizada.

No contexto deste trabalho, o método GET foi utilizado para enviar dados de forma visível na página, geralmente em situações que envolvem consultas ou visualização de rankings que não alterem a base de dados. Por outro lado, o método POST foi aplicado para operações que envolvem alterações na base de dados, como a criação ou atualização de um cliente.

Os dados submetidos no formulário são recebidos no servidor através do método `getParameter()` do objeto `HttpServletRequest`. Este método permite recuperar o valor de um campo do formulário identificado pelo atributo `name`. Por exemplo, a linha `String numeroFiscal = request.getParameter("numeroFiscal")` obtém o valor inserido pelo utilizador no campo identificado como `numeroFiscal`. Assim, no servidor, o Servlet correspondente ao formulário processa o pedido utilizando o objeto `HttpServletRequest` para recuperar os valores enviados, acedendo aos campos pelos seus nomes. Estes parâmetros são armazenados em variáveis que podem ser usadas pelo Servlet para realizar operações como validações, transformações ou alterações, em conjunto com a base de dados.

Sempre que é necessário realizar operações na base de dados, o Servlet recorre a uma classe de backend ou a métodos auxiliares. O Servlet estabelece a ligação com a base de dados, constrói comandos SQL com base nos dados recebidos e executa operações como inserir, atualizar ou consultar informações. Caso a operação seja bem-sucedida, o Servlet prepara uma mensagem de sucesso, caso contrário, apresenta uma mensagem de erro, que será exibida pela JSP na página web.

Após o processamento, o Servlet utiliza um mecanismo chamado `Request Dispatcher` para redirecionar a resposta para a página JSP sem realizar um novo pedido por parte do cliente. Na JSP, as informações enviadas pelo Servlet podem ser acedidas e apresentadas ao utilizador de forma dinâmica, visto que estas páginas permitem inserir código JAVA no meio do HTML.

Este ciclo de pedido e resposta assegura que a lógica da aplicação (processada no Servlet) se mantenha separada da apresentação dos dados na web (renderizada pela JSP). Esta separação promove uma arquitetura mais organizada e mais fácil de reutilizar.

Conclusões

Este trabalho demonstrou a importância de dominar o funcionamento e a elaboração de bases de dados. Embora interessante, o trabalho sofreu pelo pouco tempo que o grupo teve disponível para o concretizar e este fator fez com que uma das alíneas não fosse implementada na parte Web e que o suporte a imagens fosse descartado da versão final.

Alguns pontos que o grupo encontrou que poderiam ser melhorados foram os seguintes:

As Reservas poderiam ser melhor integradas com o Aluguer, visto que partilham muitos atributos. Transformar o Aluguer numa entidade fraca de reserva poderia ser a solução.

Quando o cliente reserva um veículo, o custo final embora esteja na moeda escolhida pelo cliente, não apresenta o símbolo da mesma. Embora seja um pequeno detalhe, poderia incrementar o trabalho e trazer mais algum realismo.

Outro ponto a melhorar, é quando o funcionário escolhe o carro e o condutor não se tem em atenção se estes estão disponíveis no momento do aluguer. Apenas se verifica se estão disponíveis no momento em que a reserva passa a aluguer.

Por fim, o design e a estética das páginas Web apresentadas, também poderia estar melhor, estando mais organizado e apelativo, porém mais uma vez a falta de tempo acabou por ser determinante, acabando por se dar maior prioridade ao funcionamento geral da aplicação comparativamente ao seu design.

Bibliografia

Consulta:

Trigo, P., Filipe, P., & Teófilo, A. - Slides de Sistema de Base de Dados (via Moodle)

dos, C. (2007, July 25). *artigo de lista da Wikimedia*. Wikipedia.org; Fundação Wikimedia, Inc.
https://pt.wikipedia.org/wiki/Lista_de_moedas